Engine Predictive Maintenance Business Report

Report date: 17 May 2026

Author: Debasish Das

Contents

Executive Summary

This report summarizes the predictive-maintenance investigation completed in .ipynb. The business goal is to identify engines that likely need maintenance from six sensor readings so that service teams can intervene before breakdowns create repair cost, downtime, and safety risk.

The dataset contains 19,535 engine records with no missing values or duplicate records. Faulty or maintenance-needed engines account for approximately 63% of records, so model evaluation focused on F1, precision, recall, and ROC-AUC rather than accuracy alone.

The tuned XGBoost model achieved test precision of 0.732, recall of 0.659, F1 of 0.694, accuracy of 0.633, and ROC-AUC of 0.679. This is useful as a triage signal, particularly when paired with operating thresholds and technician review, but it is not yet strong enough for fully automated maintenance decisions.

The most important business signal is Engine RPM: faulty engines have materially lower average RPM than healthy engines. Fuel pressure, lubricant-oil temperature, and coolant temperature add supporting but weaker predictive information.

Business Context and Objective

Unplanned engine failures cause avoidable repair cost, lost operating time, customer disruption, and potential safety exposure. Fleet operators and service teams need an early warning mechanism that turns routine sensor data into a practical maintenance-prioritization signal.

The objective is to classify each engine snapshot as healthy (0) or requiring maintenance (1), using RPM, lubricant-oil pressure, fuel pressure, coolant pressure, lubricant-oil temperature, and coolant temperature.

Data Registration and Quality

Methodology

The notebook created the project data folder, copied the raw engine_data.csv file into it, authenticated with Hugging Face, created the dataset repository, and uploaded the raw file. The same repository was later used as the reproducible source for train and test data.

Table 1 - Dataset and data-quality profile.

Item	Notebook result	Business implication

Rows and columns	19,535 rows x 7 columns	Enough records for a first supervised model using compact tabular features.
Predictors	Six numeric sensor fields	No categorical encoding required; all features are deployable from engine telemetry.
Target	Engine Condition: 0 = healthy, 1 = needs maintenance	Binary classification problem aligned with maintenance triage.
Missing values	0	No imputation needed in this version of the model.
Duplicate records	0	No duplicate-removal impact on sample size.
Target split	63.05% faulty, 36.95% healthy	Moderate imbalance; class weighting was used during modeling.
Registered dataset	https://huggingface.co/datasets/debasishdas1985/engine-predictive-maintenance	Supports reproducibility and external review.

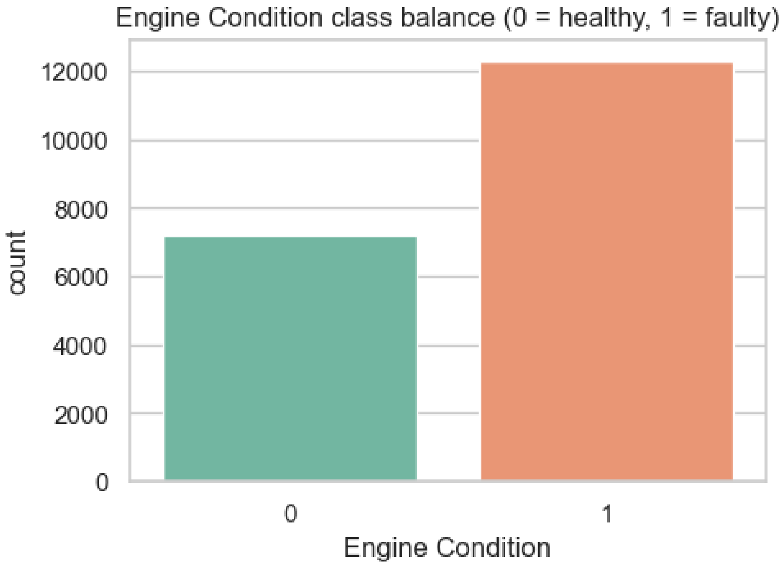


Figure 1 - Engine Condition class balance (0 = healthy, 1 = needs maintenance).

Exploratory Data Analysis

Methodology

The notebook inspected schema, missingness, duplicates, summary statistics, univariate distributions, class-wise sensor behaviour, and pairwise correlations. Histograms, boxplots, target-split boxplots, a correlation heat map, and a graph sample were used to identify patterns and modelling risks.

Key Findings

- Engine RPM is the strongest single directional signal. Healthy engines average approximately 885 RPM, while faulty engines average approximately 736 RPM.

- Faulty engines show slightly higher fuel pressure on average, which may indicate operating stress such as restricted flow or poor combustion conditions.

- Individual feature correlations are modest, and inter-feature correlations are low. This supports retaining all six sensors and using a non-linear model rather than relying on one threshold.

- Coolant temperature contains a small number of non-physical extremes; these were treated as sensor artifacts in preparation.

Table 2 - Mean sensor values by Engine Condition class.

Engine Condition	Engine RPM	Lub oil pressure	Fuel pressure	Coolant pressure	Lub oil temp	Coolant temp
0 - Healthy	884.995	3.222	6.236	2.368	78.024	78.803

1 - Needs maintenance	736.297	3.351	6.901	2.316	77.420	78.207

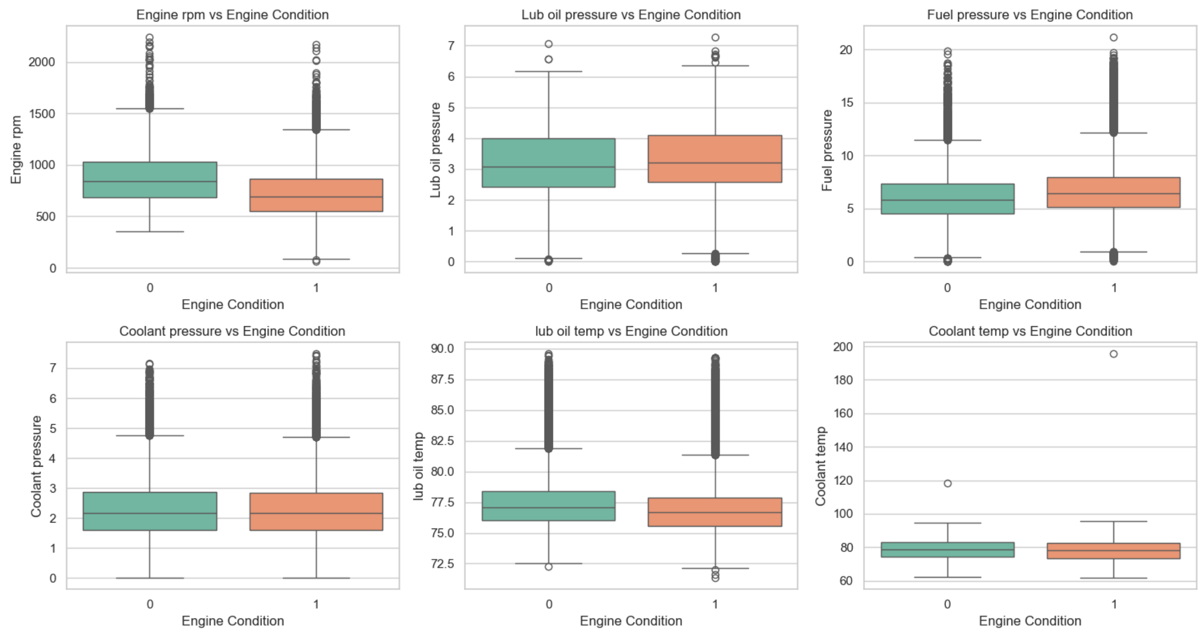


Figure 2 - Sensor values by Engine Condition, highlighting RPM separation and smaller pressure/temperature differences.

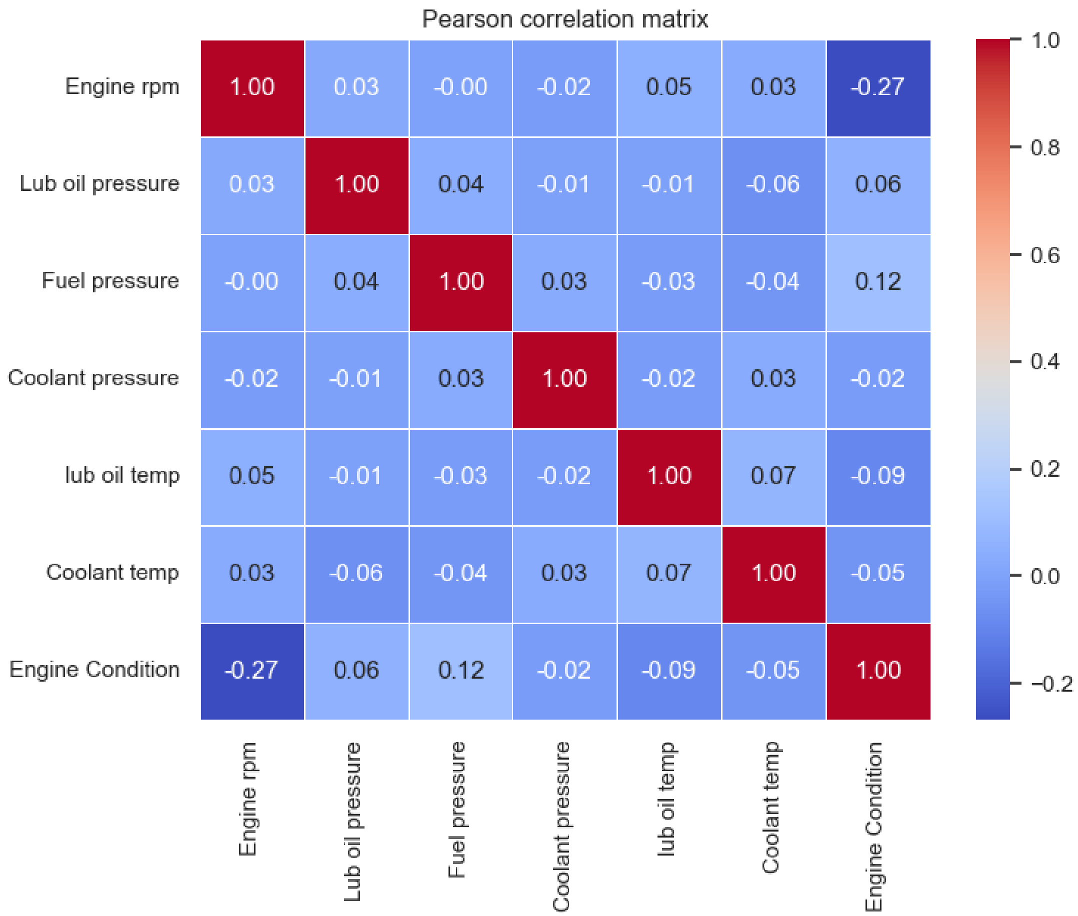


Figure 3 - Pearson correlation matrix for sensors and the target.

Data Preparation

Methodology

The notebook reloaded the raw dataset from the Hugging Face dataset repository, standardized column names to snake_case, performed defensive duplicate and missing-value removal, clipped only extreme coolant-temperature artifacts above the 99.9th percentile, and created a stratified 80/20 train-test split.

Table 3 - Data-preparation actions and resulting artifacts.

Preparation action	Notebook result	Reason
Column normalization	Converted names such as Engine rpm to engine_rpm	Makes downstream training and deployment code stable.
Duplicate and NA removal	Removed 0 rows	Confirms data quality without reducing sample size.
Outlier handling	Clipped 20 coolant_temp values above 94.06 C	Controls obvious sensor artifacts while preserving most observations.
Split strategy	15,628 train rows and 3,907 test rows	Creates a holdout test set for business-level performance assessment.
Stratification	Train/test class mix remains about 63% faulty	Prevents class imbalance from drifting between train and test.
Registered artifacts	Xtrain.csv, Xtest.csv, ytrain.csv, ytest.csv uploaded to Hugging Face	Allows reviewers or deployment pipelines to reproduce the model inputs.

Model Building and Evaluation

Methodology

The notebook trained an XGBoost gradient-boosted-tree classifier in a scikit-learn pipeline. A StandardScaler was included for a consistent preprocessing object, while XGBoost handled non-linear sensor interactions. Class imbalance was addressed with scale_pos_weight rather than resampling. GridSearchCV evaluated 48 parameter combinations using 5-fold cross-validation and F1 scoring. MLflow logged nested runs for the parameter combinations and the best model was serialized and registered on Hugging Face.

Table 4 - Best model configuration from grid search.

Parameter	Selected value
Algorithm	XGBoost binary classifier
n_estimators	200
max_depth	7
learning_rate	0.1
subsample	0.8
colsample_bytree	1.0
scale_pos_weight	0.5860
Cross-validation	5-fold GridSearchCV, 48 combinations, scoring = F1
Best CV F1	0.6949

Table 5 - Test-set performance summary.

Metric	Value	Interpretation
Accuracy	0.6330	Correct classification across both healthy and faulty records.
Precision, faulty class	0.7323	When the model flags maintenance, about 73% of those alerts are correct.
Recall, faulty class	0.6585	The model catches about 66% of engines that need maintenance.
F1, faulty class	0.6935	Balanced view of faulty-class precision and recall.
ROC-AUC	0.6792	Moderate rank-order separation between healthy and faulty engines.
Train F1	0.8930	Materially higher than test F1, indicating a generalization gap to address.

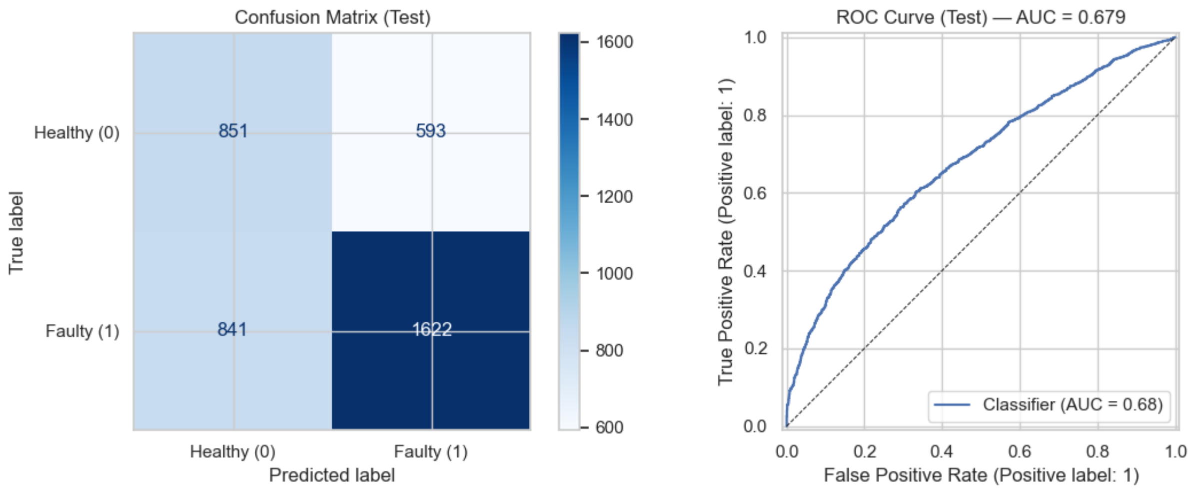


Figure 4 - Test diagnostics: confusion matrix and ROC curve for the tuned XGBoost model.

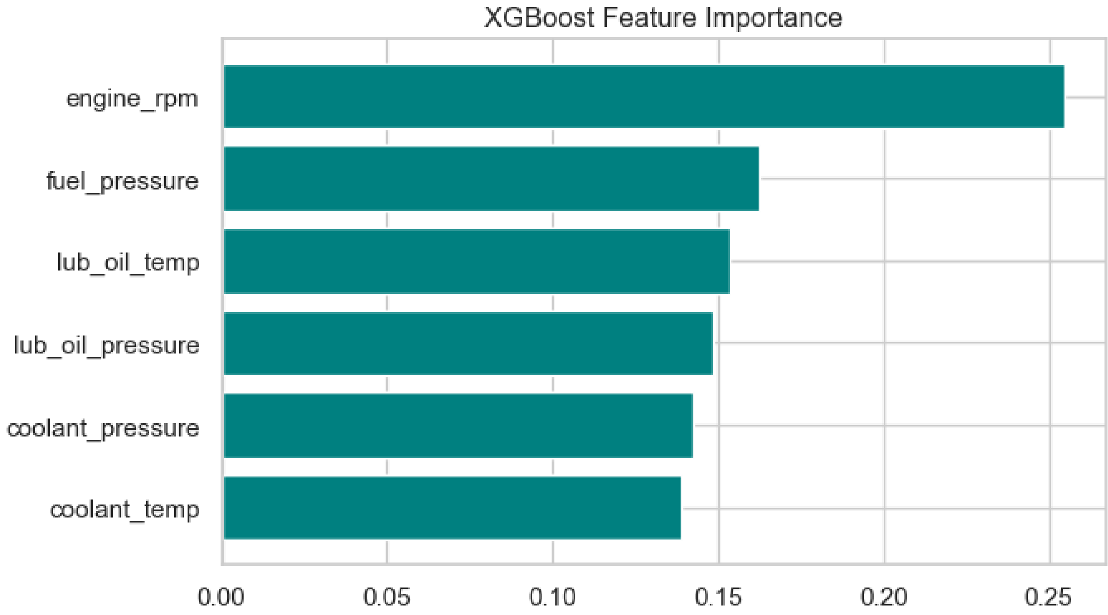


Figure 5 - XGBoost feature importance for the six sensor inputs.

Business Interpretation and Recommendations

The model is best positioned as a maintenance triage aid. Its faulty-class precision is strong enough to make alerts useful, but recall below 70% means it should not be the only protection against high-cost engine failures. The business should combine the model score with technician rules, sensor threshold alerts, and periodic model review.

- Use model probabilities to prioritize inspection queues rather than automatically scheduling repairs.

- Tune the decision threshold according to operating cost: lower the threshold when false negatives are more expensive than extra inspections.

- Collect more failure examples and maintenance outcomes to reduce the train-test performance gap and improve recall.

- Add probability calibration and threshold analysis before deployment so alerts can be interpreted as risk bands.

- Monitor live precision, recall, false negatives, and sensor drift in MLflow or a production monitoring layer.

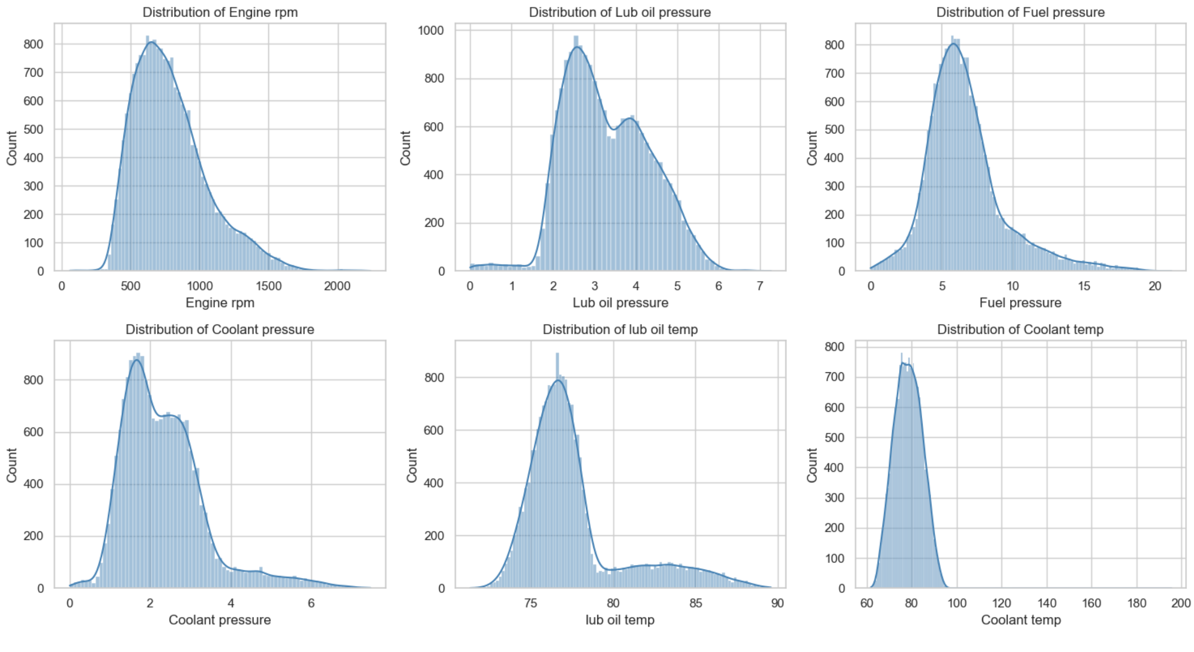
Appendix

Appendix A - Notebook Evidence and Reproducibility

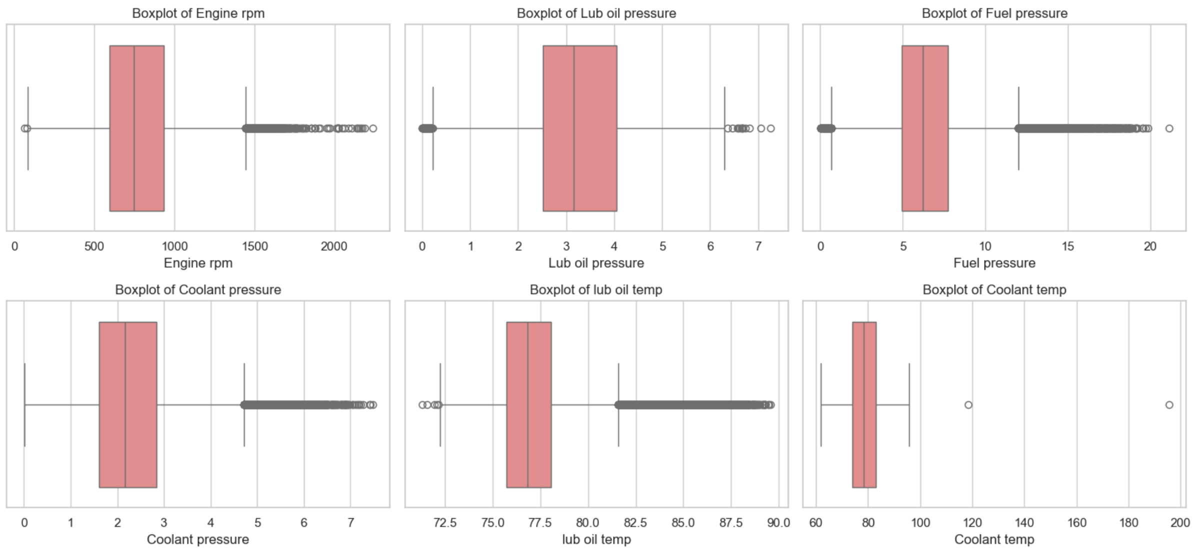
Appendix Table A1 - External resources and local artifacts.

Artifact	Location
Source notebook	project.ipynb
Raw dataset	data/engine_data.csv
Hugging Face dataset	https://huggingface.co/datasets/debasishdas1985/engine-predictive-maintenance
Hugging Face model	https://huggingface.co/debasishdas1985/engine-predictive-maintenance-model
Serialized model	best_engine_model.joblib

Appendix B - Supporting Visual Checks



Appendix Figure A1 - Univariate sensor distributions from the notebook.



Appendix Figure A2 - Boxplot-based outlier inspection by sensor.

Appendix C - Full Test Classification Report

Classification report (test set):

	precision	recall	f1-score	support
0	0.5030	0.5893	0.5427	1444
1	0.7323	0.6585	0.6935	2463
accuracy			0.6330	3907
macro avg	0.6176	0.6239	0.6181	3907
weighted avg	0.6475	0.6330	0.6378	3907